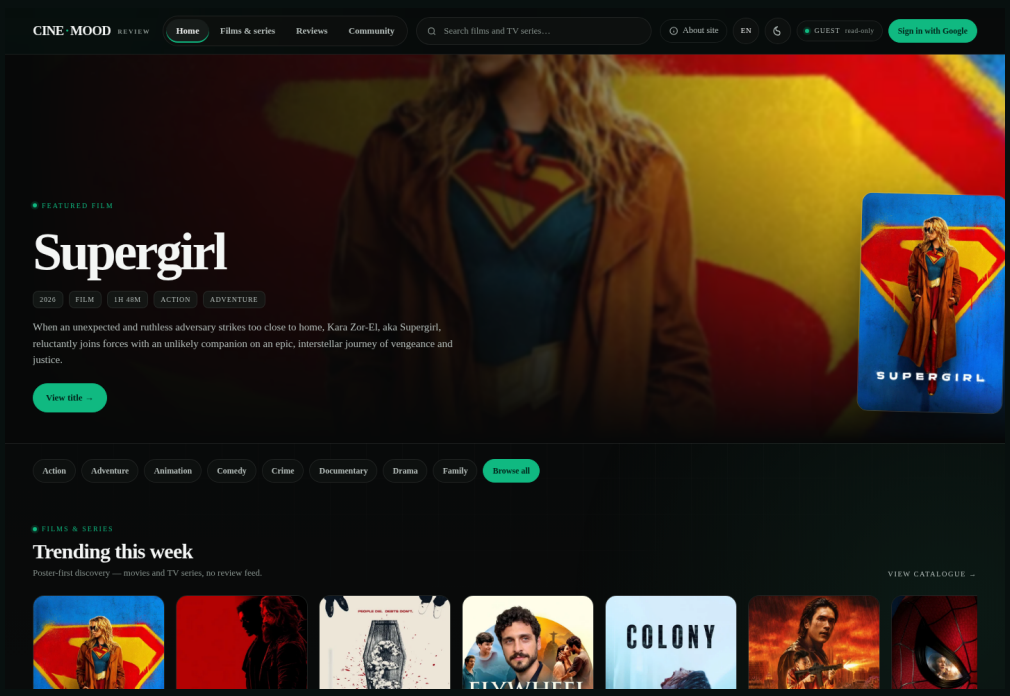
CINE·MOOD
REVIEW · RATE · DISCUSS

# Project ka complete map

Cinevood / CINE·MOOD ka product, architecture aur recent coding ko simple Hinglish mein — visuals aur code examples ke saath.

**WHAT THIS GUIDE COVERS**

App ka purpose · feature flows · TMDB + IMDbio poster fallback · database/auth · Jarvis · deployment · code file map




Repository ke existing UI screenshot ka sample. Yeh guide current source code ko explain karti hai; screenshot ko live deployment proof na samjhein.

**Prepared for Harsh Upadhyay | 31 October 2026**

01 | Pehle 2 minute mein project samjho

# Cinevood kya karta hai?

**Cinevood (CINE·MOOD)** ek cinema discovery aur community-review platform hai. Visitors films/series browse karte hain, poster/metadata dekhte hain, community verdict padhte hain; members apne ratings aur reviews publish kar sakte hain. Iska focus **reviews aur discussion** hai — yeh movie streaming, file hosting, ya download site nahi hai.

**Is document ka core**
Sabse recent coding mein poster image ka cross-source fallback add/cover kiya gaya: pehle available poster; image fail ho toh doosre provider ko try; dono se image na mile toh broken icon ke badle saaf unavailable message. Saath mein Vercel build ke do TypeScript boundary errors ko bhi narrow kiya gaya.

## Is guide mein

| Section | Aap kya seekhenge |
|---|---|
| **Product & screens** | Visitor, member aur admin ke user journeys |
| **Architecture** | Browser, Next.js server, Supabase, TMDB, IMDbio aur optional Gemini ka role |
| **Poster fallback** | UI component → server API → provider selection → final unavailable state |
| **Data, auth & Jarvis** | Review model, permissions, click-only AI review flow |
| **Deploy & testing** | Vercel/Render variables, build command, health endpoint aur known limitations |
| **Code map** | Kaun si file kya karti hai aur latest fixes ka matlab kya hai |

## Quick glossary

**Frontend** = user jo screen dekhta hai. **Backend** = server-side rules/data calls. **API** = do systems ke beech request/response. **Fallback** = pehla option fail ho toh alternate. **Build** = source code ko deploy-ready app mein compile karna.

02 | Product aur user experience

# Site ka flow: discover → detail → verdict

Homepage poster-first hai: featured title, genres aur trending/new-arrival rails. Search suggestions aur full results se local catalogue aur external discoveries tak ja sakte hain. Movie/series detail page metadata, score, cast, synopsis aur reviews dikhata hai. Review interaction ke liye login zaroori; signed-out visitor read-only mode mein browse kar sakta hai.

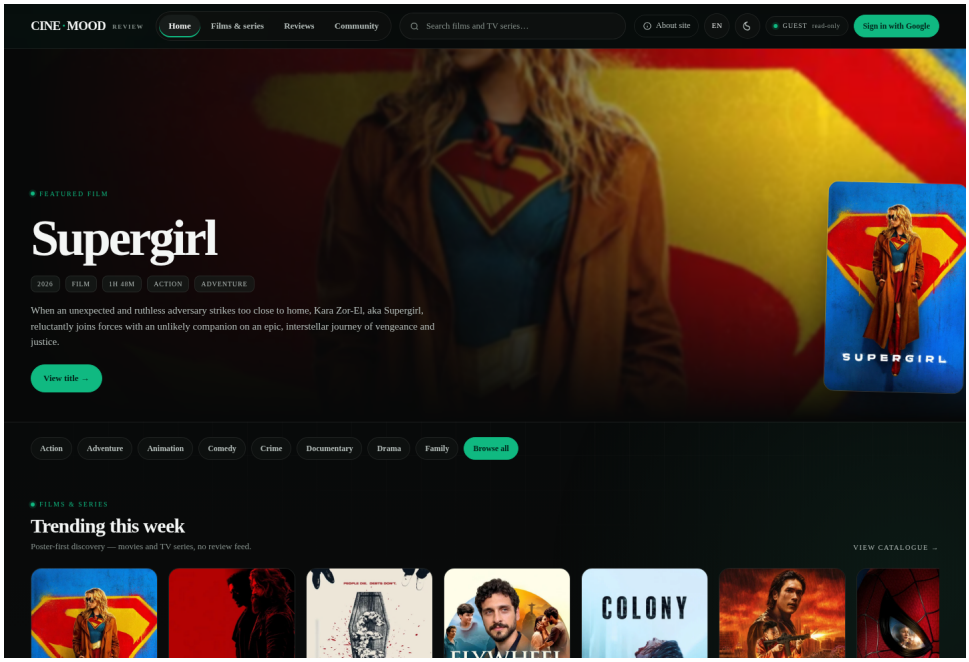



Figure 1. Poster-led homepage ka repository screenshot: featured title, genre chips aur title cards. Poster fallback yahin ke shared card component ko cover karta hai.

## Main product areas

| Area | Meaning |
|---|---|
| **Discovery** | Films aur TV series ko poster/title/genre ke through explore karna. |
| **Title detail** | Metadata + rating context + cast + similar titles + reviews ek page par. |
| **Community** | 10-point rating, spoiler-marked review, like aur comments. |
| **Personal dashboard** | My reviews, watchlist, profile aur basic stats. |
| **Admin control room** | Moderation, catalogue/import, editorial controls, audit/analytics. |
| **Language/accessibility** | English/Hindi/Hinglish editions; guest read-only browsing; keyboard search support. |

03 | Screens par poster kahan nazar aata hai?

# Ek shared image component, multiple surfaces

Search dropdown, search results, homepage rails, detail page, review links aur discovery pages alag-alag UI hain. Agar har screen apni image-error logic likhe toh behavior inconsistent hota. **PosterImage** ek shared component hai jo image loading/fallback/unavailable state ko centralize karta hai.

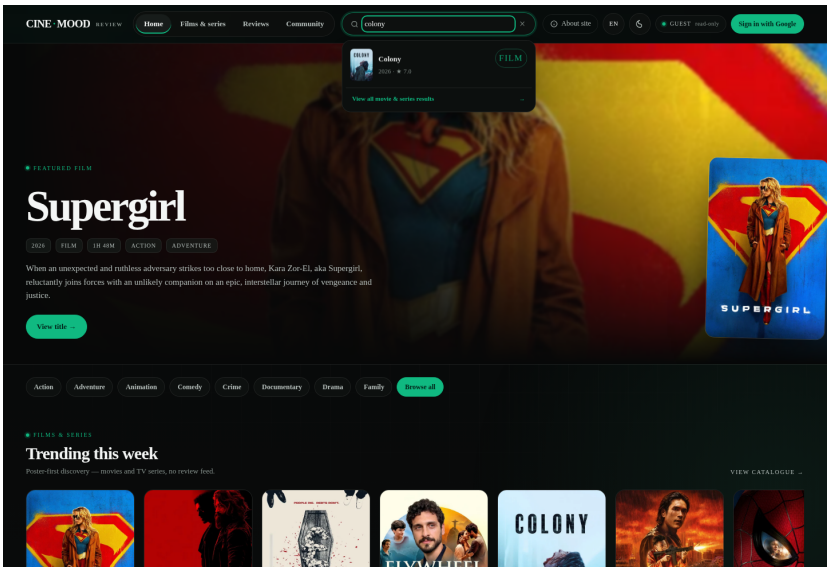



Figure 2. Search dropdown screenshot. Local result par stored poster, external result par IMDbio cover ho sakta hai; failure par doosra provider try hota hai.

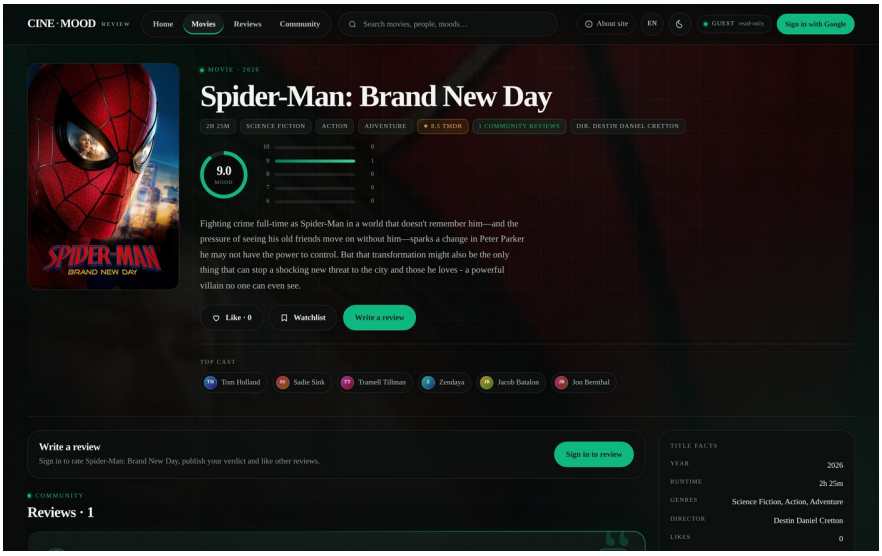


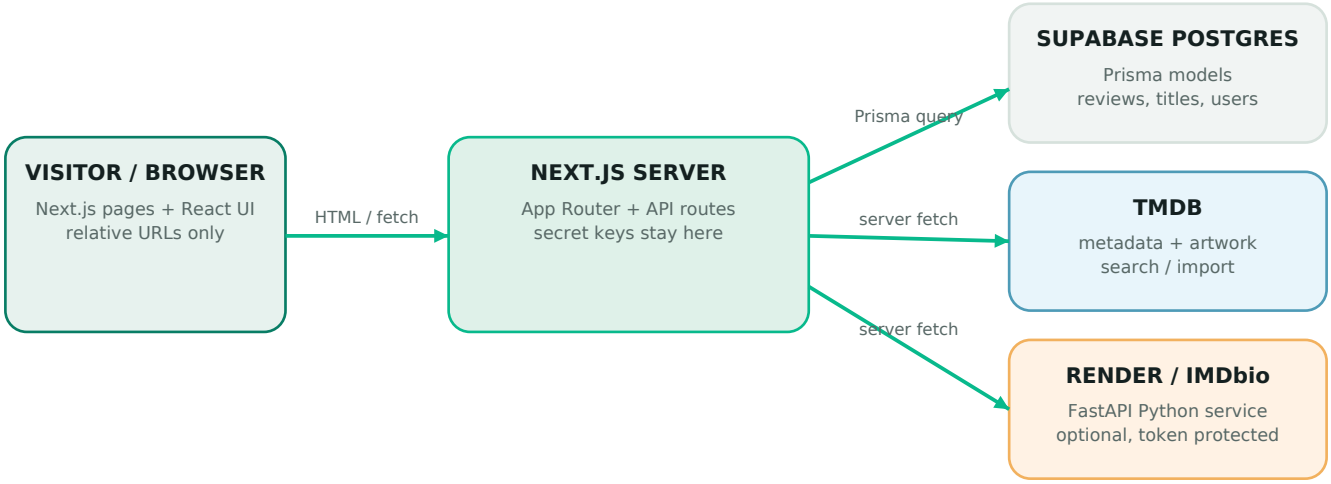

Figure 3. Title detail screenshot. Poster yahan zyada prominent hai; stored title poster primary aur fetched IMDbio cover secondary source ho sakta hai.

Covered code surfaces: **PosterCard** (home/catalogue), **SearchBox** dropdown, **/search** result cards, **/movies/[slug]** detail, **/explore** TMDB discovery, **/imdb/[id]** IMDbio discovery, review feed/cards, dashboard recommendations.

04 | Technical architecture

# Browser se data provider tak

Next.js App Router same project mein server-rendered pages aur API handlers deta hai. Browser relative route, jaise `/api/imdb/poster`, call karta hai. Private provider credentials server par rehte hain; server TMDB/IMDbio/Gemini ko call karke safe response bhejta hai.




Optional AI: Gemini is also called from server-side routes; its key is never a browser variable.

Diagram: request boundary aur responsibility map. Arrows ka matlab direct browser-to-provider call nahi hai; provider requests server-side hoti hain.

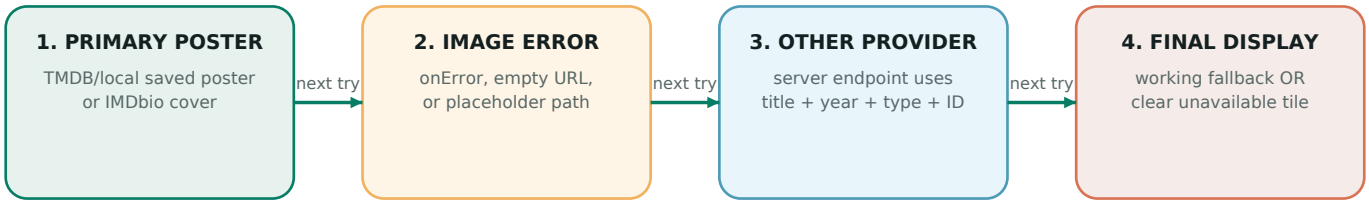
## Technology ka simple matlab

| Layer | Repo mein role |
|---|---|
| **Next.js 15 + React 19** | Pages, route handlers, server/client components. |
| **TypeScript** | Props, input, API shapes par compile-time checks. |
| **Prisma + Postgres** | Title/review/user data; Supabase Postgres runtime database. |
| **TMDB** | Catalogue/search/import metadata, ratings aur poster/backdrop URLs. |
| **IMDbio FastAPI on Render** | Optional IMDb details/search/reviews; Next.js is service ko HTTPS se call karta hai. |
| **Auth.js** | Google member login; alag server-checked admin credentials/session. |
| **Gemini (optional)** | Jarvis questions/review drafts/translations; API key server-side. |

05 | Poster fallback: exact behavior

# Aapki request ko code ka jawab

Ab primary poster render hota hai. Agar primary URL empty, placeholder, ya image request fail ho, component alternate provider ke liye same title ki lookup karta hai. Provider direction source ke hisaab se hoti hai; ek hi direction har case par force nahi hoti.




**Direction rule:** TMDB/local primary -> IMDbio secondary; IMDbio discovery primary -> TMDB secondary.

Both providers are resolved on the server. The browser receives only a poster URL or null, never API credentials.

When neither returns a usable URL: PosterImage replaces the broken image with an accessible unavailable message.

Matching is cautious: media type must match; title is normalized; year can differ by at most one when both sides have a year.

Image error handler + server lookup + final UI: yeh teen layers milkar fallback banati hain.

## Provider direction ka example

| Case | Primary | Fallback attempt |
|---|---|---|
| **Local catalogue / TMDB-backed poster** | Title.poster (stored poster URL) | IMDbio — saved IMDb ID available ho toh direct, warna title/year/type matching. |
| **IMDbio search result** | IMDbio cover URL | TMDB multi-search, exact normalized title + media type + compatible year. |
| **Title detail with prefetched IMDbio cover** | Stored poster primary; `secondarySrc` can hold IMDbio cover | Known secondary is used first; missing ho toh configured fallback route. |
| **No valid image in either source** | Broken/empty image | Text tile: “Movie thumbnail unavailable” / “Series thumbnail unavailable”. |

**Important meaning**

“Thumbnail unavailable” ka matlab sirf artwork fetch nahi hua. Film/series record ko delete ya hidden nahi kiya jaata; title, review aur metadata page phir bhi accessible reh sakte hain.

Fallback poster database ko overwrite nahi karta. Yeh on-demand display behavior hai: original stored poster record jaisa tha waisa rehta hai.

06 | Frontend code ka breakdown

# PosterImage.tsx: image ke states

File: `components/PosterImage.tsx`. Yeh React Client Component hai kyunki image ka `onError` browser mein fire hota hai aur fallback request ke baad UI state change hoti hai.

```
// Simplified current behavior (conceptual excerpt)
const [imageUrl, setImageUrl] = useState(src || '');
const [failed, setFailed] = useState(false);
const fallbackAttempted = useRef(false);

<img
  src={imageUrl || '/posters/placeholder.svg'}
  onError={() => {
    if (fallbackAttempted.current) setFailed(true);
    else void findFallbackPoster();
  }}
/>
```

## Is snippet ka matlab

- **imageUrl**: abhi ka selected image URL; pehle source se start hota hai.
- **onError**: browser image download/decode fail hone par trigger karta hai.
- **fallbackAttempted**: ek image ke liye repeated infinite retries ko rokta hai.
- **failed**: dono source path ke baad unavailable tile dikhane ka signal.
- **secondarySrc**: detail page jaise context mein pehle se fetch hua alternate URL; isse extra lookup bach sakta hai.

When no `src` or placeholder path appears, effect fallback lookup start karta hai. Fallback load bhi fail ho toh `failed` true hota hai; component `<img>` ko accessible `<div role="img">` + message se replace karta hai. Lazy/eager loading parent UI ki priority ke hisaab se pass hoti hai.

```
// Fallback request browser se same-origin route par:
GET /api/imdb/poster?title=...&year=...&mediaType=movie&source=imdbio

// Expected safe response:
{ "coverUrl": "https://..." }  // ya { "coverUrl": null }
```

**Security point**

Component mein IMDbio token ya TMDB key nahi hoti. Browser sirf title context aur same-origin route ko request karta hai; credentials server environment mein hote hain.

07 | Server API: provider choose karna

# /api/imdb/poster ka kaam

File: app/api/imdb/poster/route.ts. Yeh route fallback lookup ko server side par rakhta hai. Request mein title, year, media type, optional IMDb ID aur fallback source aate hain. Route input validate karta hai, per-IP rate limit lagata hai, provider lookup karta hai aur poster URL/null return karta hai.

```
// Simplified branch logic
if (source === 'tmdb') {
  // TMDB multi-search; match title + movie/tv + year
  coverUrl = matchingResult ? tmdbImage(posterPath, 'w500') : null;
} else {
  // IMDbio exact catalogue match or saved IMDb ID
  coverUrl = (await getImdbTitleForCatalogue(...))?.coverUrl ?? null;
}
return NextResponse.json({ coverUrl });
```

## Lookup safeguards

- **IMDb ID validation:** accepted ID ka shape `tt` + digits hona chahiye.
- **Title matching:** normalize punctuation/case; wrong movie/series artwork ki chance kam karna.
- **Year check:** dono taraf year ho toh +/- 1 saal ke andar match; missing year ko hard block nahi banaya.
- **Media type:** movie ko movie, TV ko TV result ke saath match karta hai.
- **Rate limit:** shared `LIMITS.search` bucket (90 requests/minute per IP). 429 aane par client gracefully unavailable state dikha sakta hai.
- **Cache:** response par CDN cache 1 ghanta aur stale-while-revalidate 1 din tak.

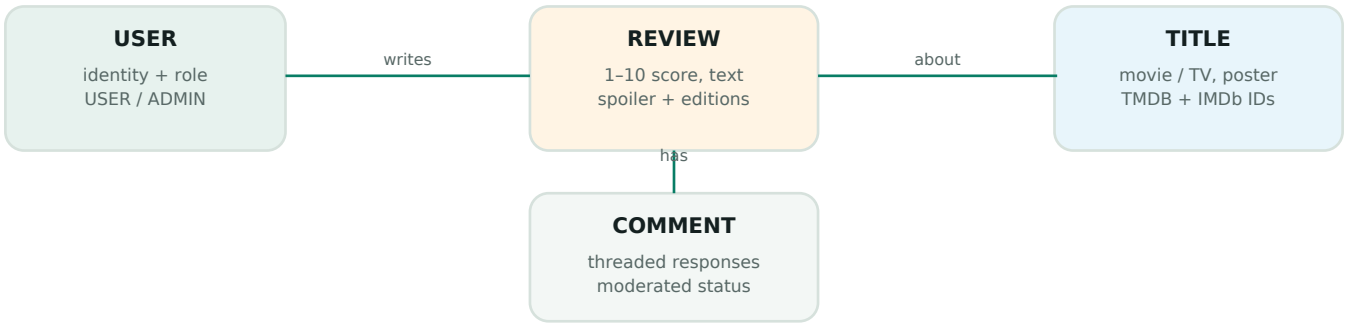
Linked server modules: lib/imdbio.ts is marked server-only; lib/tmdb.ts reads TMDB credentials from server env and supports proxy-safe requests.

08 | Data, reviews aur access rules

# Database ka simple model

Prisma schema mein **Title** film/series record hai; **Review** title aur author ka verdict hai; comments/likes title ya review se relate hote hain. Poster fallback ke liye existing Title fields—poster, mediaType, imdbId—read kiye jaate hain; koi nayi DB migration required nahi.

Support tables not shown: ReviewLike, TitleLike (LIKE/WATCHLIST), RateLimit, AdminLog, PageView.




Schema ka simplified relationship diagram. `Review.userId` nullable ho sakta hai, jaise legacy editorial content ke liye.

## Data/permissions ka meaning

| Concept | Behavior |
|---|---|
| **Title.mediaType** | Schema mein string hai; UI boundary par value ko 'movie' ya 'tv' union mein narrow kiya jaata hai. |
| **Review.rating** | 1–10 score; review title se linked; per user/title uniqueness defined. |
| **spoiler/status** | Spoiler reveal UI control karta hai; review status publish/pending/hidden workflow ka part hai. |
| **Guest** | Read-only browse; interaction ke liye sign-in CTA. |
| **Member** | Google sign-in; reviews, likes, comments, dashboard, watchlist. |
| **Admin** | Separate admin auth check; moderation/editorial/import/analytics controls. |
| **Audit & protection** | RateLimit shared table + memory layer; admin mutations AdminLog mein audit ho sakti hain. |

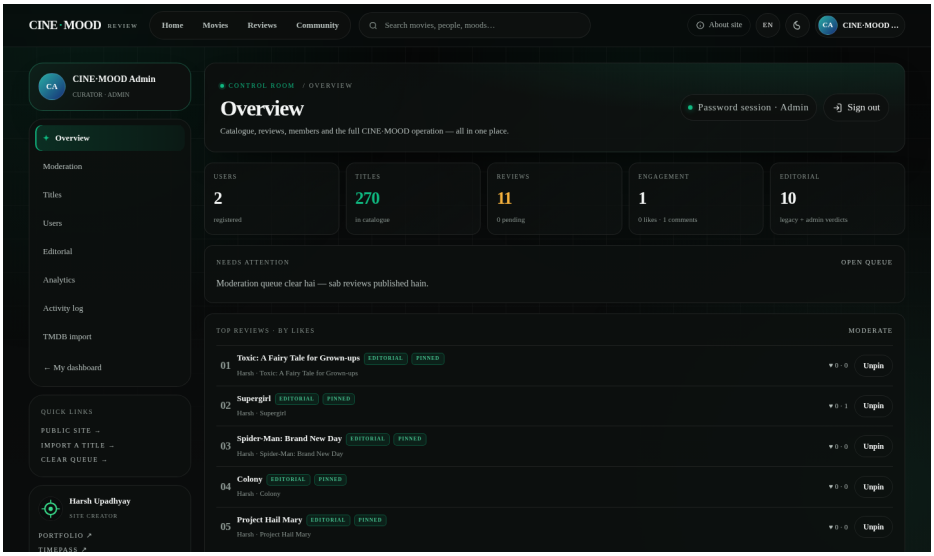



Figure 4. Admin dashboard screenshot: site operations ko public member experience se alag rakha gaya hai.

09 | Jarvis: AI ko bounded rakhna

# Click-only, grounded, editable

Jarvis app ka AI assistant/editorial feature hai; yeh alag concern hai poster fallback se. Review generation user/admin ke click par start hoti hai—automatic catalogue-wide batch nahi. Published review already maujood ho toh auto-review flow manual review ko replace nahi karta.

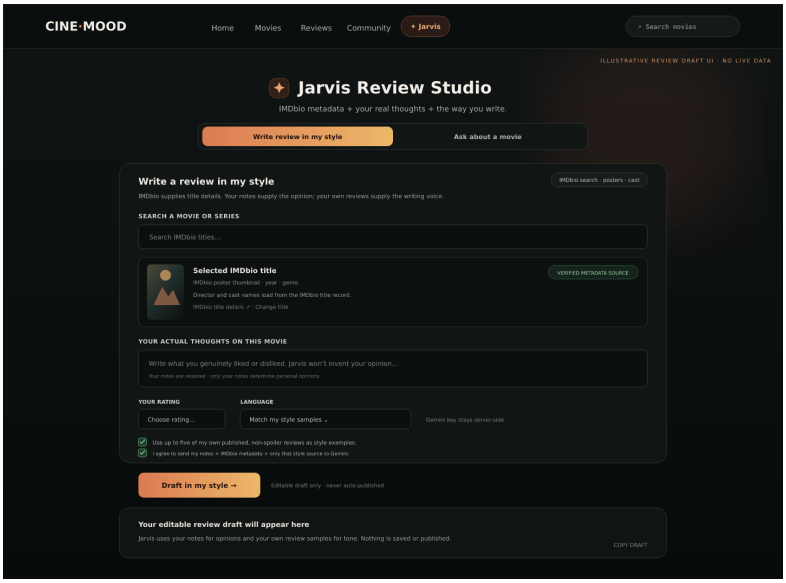



Figure 5. Repository ka labelled illustrative Jarvis Review Studio UI. Yeh screenshot “illustrative” hai; live state ka proof nahi.

## Review generation ka flow

| Step | Kya hota hai |
|---|---|
| **1. Trigger** | User “Jarvis Review” button click karta hai; client POST /api/jarvis/auto-review bhejta hai. |
| **2. Grounding** | Server title facts / IMDbio metadata aur available non-spoiler reviews gather karta hai. |
| **3. Style** | Owner ke published reviews style examples ke roop mein use kiye ja sakte hain; user-provided thoughts/consent where applicable. |
| **4. Save** | Generated bilingual headline/body aur Jarvis ka distinct 1–10 verdict review record mein save hota hai. |
| **5. Display** | Hinglish/English switch se editions dikhte hain; source/disclosure AI nature ko clear karta hai. |

**AI boundary**
Gemini key server-side. Jarvis ko facts grounded rakhne, manual reviews preserve karne, aur user ke bina automatic mass-generation na karne ke liye request/route checks diye gaye hain.

10 | Repository file map: “yeh file kyun hai?”

# Coding ko source files se connect karo

| Path | Responsibility / plain-English meaning |
|---|---|
| **app/page.tsx** | Homepage: featured hero + catalogue rails; PosterImage hero and PosterCard rails. |
| **app/movies/[slug]/page.tsx** | Local title detail: DB facts, IMDbio facts, reviews, similar titles; TMDB/local primary + IMDbio secondary poster. |
| **app/search/page.tsx** | Full results screen; title/source context ke saath PosterImage. |
| **components/SearchBox.tsx** | Header dropdown, debounced results, keyboard navigation aur poster previews. |
| **components/PosterCard.tsx** | Reusable card for catalogue/home poster rails. |
| **components/PosterImage.tsx** | Client-side primary image, one fallback attempt, loading state, unavailable state. |
| **app/api/imdb/poster/route.ts** | Server-only resolver: IMDbio ya TMDB fallback; validation, match logic, rate limit, cache. |
| **lib/imdbio.ts** | IMDbio server client: auth header, timeouts, response parsing, exact title resolver. |
| **lib/tmdb.ts** | TMDB API search/details/image helpers; API key stays on server. |
| **lib/queries.ts** | Prisma read queries; PosterTitle mein optional imdbId add hua so fallback direct IMDb lookup kar sake. |
| **app/reviews/page.tsx + [id]/page.tsx** | Review screens ke compact poster placements; mediaType ko component union se align kiya. |
| **app/globals.css** | Shared visual state: unavailable tile and smaller card/rail dimensions. |
| **prisma/schema.prisma** | Persistent data model: Title, Review, User, Comment, likes, RateLimit, AdminLog, PageView. |
| **imdbio-service/main.py** | Python FastAPI wrapper around IMDbio; Render par health endpoint /health. |

## How to navigate code

UI bug? Pehle component/page dekho. Data mismatch? `lib/queries.ts` aur Prisma schema compare karo. External provider issue? `lib/imdbio.ts` / `lib/tmdb.ts` dekho. Browser ko secret chahiye lag raha ho? Aksar sahi design yeh hai ki secret ko client mein na bhejein; server route call kare.

11 | Deployment aur environment variables

# Vercel + Render + Supabase

Current deployment model mein Next.js app Vercel par, IMDbio Python service Render par, aur data Supabase Postgres par hai. Existing report mein Render base URL `https://cinenvood-hn7a.onrender.com` user-reported hai; health route `/health` intended route hai. Root `/` par FastAPI 404 hona expected ho sakta hai—health check `/health` se karo.

## Vercel project → Settings → Environment Variables

| Variable name | Value ka rule | Scope |
|---|---|---|
| **IMDBIO_API_URL** | Actual Render service URL, trailing slash ke bina. Example: `https://cinenvood-hn7a.onrender.com` | Server-side only; Production/Preview as needed |
| **IMDBIO_API_TOKEN** | Render ke `IMDBIO_SERVICE_TOKEN` ke barabar exact same secret. Is PDF/chat/repo mein token likhna nahi. | Server-side only; no NEXT_PUBLIC_ |
| **TMDB_API_KEY** | Existing TMDB API key or v4 read token; existing TMDB features ke liye. | Server-side only |
| **DATABASE_URL / DIRECT_URL** | Supabase pooled runtime URL / direct migration URL, project setup ke hisaab se. | Server only |
| **GEMINI_API_KEY** | Optional Jarvis; user consent / server route behavior ke liye. | Server-side only |

**After environment change**

Variables save karne ke baad Vercel ka naya deployment/redeploy required hota hai. Render free service idle ke baad sleep kar sakti hai; first request slow ho sakti hai. `render.yaml` mein service root `imdbio-service`, Python start command aur `/health` configured hain.

Never paste secret values into frontend code, `NEXT_PUBLIC_*`, screenshots, docs, Git commits, or logs. Token ko Vercel dashboard aur Render environment ke beech hi rakho.

12 | Is kaam mein exact coding kya hui?

# Change ledger — code ka “matlab”

| Change | Code effect | User ko kya dikhega |
|---|---|---|
| **Shared PosterImage** | Primary src/secondary src, onError, one-shot provider fallback, missing/loading UI. | Broken image ke badle alternate poster; warna clear unavailable message. |
| **Poster fallback route** | GET /api/imdb/poster validates fields, rate-limits, server-side TMDB/IMDbio lookup karta hai. | Search result ya image failure ke baad silent, same-site lookup. |
| **IMDb ID data propagation** | PosterTitle selection/mapping mein imdbId add; review query title info extend. | IMDbio direct detail lookup zyada accurate; title search fallback bhi possible. |
| **Screens wired to shared component** | Home/catalogue/search/title/reviews/explore/IMDb/dashboard. | Same behavior multiple places; one code path. |
| **Fallback CSS** | Aspect ratio, responsive compact variants, accessible alt/role. | Card layout nahi toot-ta; unavailable state clearly styled. |
| **TypeScript boundary fixes** | DB mediaType string ko `mediaType === "tv" ? "tv" : "movie"` mein normalize. | Build type props align; DB schema ko unnecessarily migrate nahi kiya. |

## Vercel build errors ka context

Vercel ne pehle `app/reviews/[id]/page.tsx` aur phir `app/reviews/page.tsx` par same type mismatch report kiya: Prisma schema ka `mediaType` plain string hai, jab reusable component ko sirf `"tv" | "movie"` allowed hai. Dono call-sites par explicit conditional normalization add ki gayi. Iska matlab data ko cast karke chupana nahi; expected two values ko runtime par select karna hai.

```
mediaType={record.mediaType === 'tv' ? 'tv' : 'movie'}
// Build ko narrowed literal milta hai; runtime DB value ko safe default milta hai.
```

**Build verification: honest status**

Sandbox mein `npm run build` Prisma engine binary ke download TLS handshake (`binaries.prisma.sh`) par ruk gaya, Next.js compile tak nahi pahucha. Isliye PDF full production build green hone ka claim nahi karti. Latest Vercel deploy ko current branch snapshot par check karein; naya error aaye toh generic BUILD_UTILS line se upar ka pehla Type error/Error dekhein.

13 | Test checklist aur troubleshoot

# Manual QA: poster fallback kaise verify karein?

| Test | Expected result |
|---|---|
| **Valid TMDB/local poster URL** | Primary image render; fallback API call ki zaroorat nahi. |
| **Temporarily break primary image URL in a test record** | PosterImage calls `/api/imdb/poster` with source and title context. |
| **IMDbio has exact matching cover** | IMDbio image appears; no token browser network request mein nahi dikhna chahiye. |
| **IMDbio-first search result poster fails** | Fallback route TMDB exact title/type/year search karta hai. |
| **No provider has a matching poster** | Poster rectangle displays “Movie/Series thumbnail unavailable”; page/card remains usable. |
| **Invalid / mismatched title metadata** | No risky arbitrary title match; endpoint returns null and UI degrades gracefully. |
| **Route rate-limited / service asleep** | No page crash; unavailable state; retry after service wakes/limit resets. |

## Deploy/build troubleshooting

- **Only `BUILD_UTILS_SPAWN_1`?** Wrapper hai. Uske upar actual Type error ya Prisma error find karo.
- **IMDbio poster absent?** Vercel mein `IMDBIO_API_URL` aur token value Render secret se match hon; latest deployment redeploy hua ho; Render `/health` check karo.
- **TMDB fallback absent?** Server-side `TMDB_API_KEY` configure ho; title/year/mediaType match ho; TMDB search result ke paas poster_path ho.
- **Generic unavailable tile?** Dono providers se usable image nahi mila ya request fail/rate-limit hui. Title data ko artwork se alag samjho.
- **Build still red?** Full first compiler error share karo. Ek error fix ke baad Next.js next file ka error dikha sakta hai.

14 | Mini glossary + handover

# Technical lafz, seedhe matlab

| Term | Simple meaning |
|---|---|
| **Component** | Reusable UI piece; example: poster card or PosterImage. |
| **Props** | Parent component se child ko bheja gaya data/options. |
| **State** | Client UI ki current condition—loading, image shown, or unavailable. |
| **API route** | Next.js server endpoint jo request validate karke provider/DB se baat karta hai. |
| **Exact match** | Title normalize karke type/year ke saath compare; similar naam ko blindly select nahi karna. |
| **Rate limit** | Ek client/IP requests ko limit karna; abuse/spikes se protect. |
| **Server-only secret** | API key/token jo browser bundle/network response mein nahi aana chahiye. |
| **Prisma generate** | Schema se type-safe client generate; local build ko Prisma engine binary download ki zaroorat ho sakti hai. |
| **Redeploy** | Updated env/code ke saath Vercel ko naya build aur deployment karna. |

## Project ki current direction

- TMDB ko remove nahi kiya gaya; existing catalogue/discovery/import use-cases continue.
- IMDbio optional complementary source hai; private token sirf server environments mein.
- Poster fallback UI-level display solution hai—DB migration ya poster URL overwrite nahi.
- AI review generation user-triggered aur distinct editorial item; existing manual reviews preserve.
- Next operational step: Vercel par newest code/environment variables ke saath production build verify karna.

**Short version**

Cinevood ka goal: cinema titles discover karna, community ke honest reviews padhna/likhna. Recent poster coding ka goal: artwork provider fail ho toh doosre provider ko try karna—aur fail state ko broken icon ke bajay clearly batana.

ADDENDUM | POSTER GUIDE KA LATEST BEHAVIOR

# Jarvis review + YouTube trailer: ab user ko kya dikhega?

Yeh addendum pehle 15-page guide mein bataye project ko clarified user request ke mutabik update karta hai. Iska focus sirf title-detail experience aur saved/reusable behavior hai.

## 1. Jarvis Review ab kahan hai?

Movie/series detail page ke Reviews section mein pehle heading, phir Jarvis review panel, phir member ka review composer aur community cards aate hain. Hero se Jarvis Review link isi panel par scroll karta hai. Ask Jarvis a question alag Q&A feature hai—dono buttons ka kaam alag label kiya gaya.

| Situation | Page behavior |
|---|---|
| **Jarvis review pehle se saved hai** | Server database se exact title ka published Jarvis record read karta hai. Review bina kisi generation request ke turant dikh jaata hai; Hinglish/English tab aur Jarvis rating saath rahte hain. |
| **Is title par saved Jarvis review nahi** | Usi Reviews area mein click-only “Jarvis Review” button dikhta hai. Click ke baad chosen titleId/IMDb ID se review generate hota hai aur usi title record ke saath save hota hai. |
| **Next visitor** | Wahi saved row server se read hoti hai; generate button phir nahi dikhaya jaata. Backend cached record pehle check karta hai. |
| **Manual/editorial review** | Existing manual review ko overwrite nahi kiya jaata. Jarvis ka saved review aur human review alag records/voices hain. |

## 2. Trailer ab kaise dhoondhta hai?

Saved/curated English ya Hindi YouTube ID ho toh wahi first choice. Saved link na ho, ya user doosra option maange, toh detail-page finder title ka TMDB-linked YouTube trailer/teaser check karta hai. Agar TMDB par video na mile, optional YouTube Data API search broader match dhoondhti hai.

| Order | Source / action | Agar fail ho |
|---|---|---|
| **1** | Saved admin English/Hindi trailer IDs; existing TrailerModal. | Detail page still offers Find another trailer. |
| **2** | TMDB videos endpoint using stored tmdbId; otherwise exact title + year + movie/TV search. | YouTube Data API v3 query tries precise then broader title trailer search. |
| **3** | Selected YouTube iframe player. | Embed blocked ho toh selected video ka direct YouTube watch link; no source ho toh YouTube search link. |

## 3. Kaun si settings chahiye?

| Variable | Purpose |
|---|---|
| **TMDB_API_KEY** | Existing server-side key se TMDB videos/metadata. TMDB par video linked ho toh YouTube key ke bina bhi trailer candidate mil sakta hai. |
| **YOUTUBE_API_KEY** | Optional broader title-based discovery aur Admin → Titles search. Key missing ho, TMDB result na ho, toh user ko YouTube search link diya jaata hai. |
| **GEMINI_API_KEY** | Jarvis draft generation ke liye server-side; saved review display karne ke liye har visitor par generation nahi hoti. |

Key code paths: `app/movies/[slug]/page.tsx` (Reviews section + saved Jarvis lookup), `components/JarvisAutoReview.tsx` (saved vs click-to-generate UI), `app/api/jarvis/auto-review/route.ts` (cache/save), `app/api/imdb/trailer/route.ts` and `lib/tmdb.ts` (TMDB-first trailer resolver), `components/ImdbTrailer.tsx` / `components/YouTubePlayer.tsx` (picker/player/direct link).

Security: no API token is printed here. TMDB, YouTube and Gemini credentials remain server-side. Guide addendum prepared 31 Oct 2026.